

Treasury Register — Data Sources & Path to Real

Internal note for Anil Passi and Goutham Sridharan · September 2026 · Fusion Practices

The Treasury Register is a control layer that sits above Oracle Fusion. It runs a proposed → approved → executed → confirmed → settled → matured → closed lifecycle, gates every path to book through six deterministic checks (counterparty active, instrument permitted, entity limit, group limit, tenor band, concentration), and layers four AI features on top (rank, read, classify, narrate) — every one of which is proposed by the model and disposed by deterministic code.

What it does NOT do: cash positioning, cash forecasting, or the movement of money. Positioning and forecasting are EPM's job — Oracle also shipped their own Cash Processing Agent for that in Release 26C. Movement is Fusion's job. The Register is the decision-and-control layer in between: given a surplus, which permitted counterparty and instrument, subject to the six checks.

This note is the honest answer to the question Anil and Goutham asked on the Sep 7 call: where does the data come from? It covers the three areas — cash pooling and bank data, rates, and accounting events — with the real Oracle / Bloomberg / bank endpoint named for every field, what's currently stubbed for the prototype demo, and what the concrete step to make it real looks like.

Table A — Cash Pooling & Bank Data

Field on screen	Real source in production	Current in demo	Step to make it real
Pool structure (header, hierarchy)	Fusion Cash Management REST — cashPools	Stub JSON	OIC + ERP Adapter GET cashPools
Pool members	Fusion — cashPoolMembers	Stub JSON	OIC GET on cashPoolMembers
Internal transfer submission	Fusion — POST cashBankAccountTransfers	Stub POST returning fake request id	Wire OIC invoke to cashBankAccountTransfers
Transfer status lifecycle	Fusion — nine states (New → Validated → ... → Settled / Failed)	Timer-driven state machine	Poll cashBankAccountTransfers on 30s cadence
Bank balance (per account)	No REST — BICC extract or BI Publisher only	Stub value with 15-min as-of timestamp	Scheduled BICC extract → Register cache
Bank statement	No REST — FBDI or BICC only	Not shown	FBDI ingestion to a reconciliation table
Cash position	No REST — Essbase cube only	Not shown	Out of scope for Register — EPM territory
Cash forecast	No REST — Essbase cube only	Not shown	Out of scope for Register — EPM territory
Notional pool arrangement	Fusion — cashPools (definition only; no money moves)	Stub definition + tooltip	OIC PATCH on cashPools

Two hard gaps. Oracle exposes no REST for bank statements/balances (BICC/BIP/FBDI only) and no REST for cash positions/forecasts (Essbase cube only). These are Oracle-side gaps, not ours.

Table B — Rates

Field on screen	Real source in production	Current in demo	Step to make it real
FX spot pre-fill	Bloomberg BGN via B-PIPE, or LSEG Real-Time via WebSocket	Stub value with as-of timestamp	Bloomberg B-PIPE contract or LSEG RDP/RTO

Field on screen	Real source in production	Current in demo	Step to make it real
FX forward pre-fill	Bloomberg FRD via B-PIPE, or LSEG forward curves	Stub value	Same feed, forward-curve entitlement
FX firm rate on booking	Bank RFQ/booking API (DBS, BofA) or 360T / FXall / FXGO	Simulated quote id + 90s expiry	Bank-direct API + mTLS + IP allowlisting
Deposit rate pre-fill	No market-data feed exists — last-known negotiated rate for counterparty	Register-held negotiated rate memory	Persist negotiated rates in Register
Deposit rate on ticket	Bank portal, dealer chat, or 360T MM / ICD	Manual override field (prominent)	Keep manual — this IS the workflow
As-of timestamp	Every feed carries this	Rendered on every rate	Real feed replaces stub source
Rate quality flag (indicative / firm)	Every feed carries this	Rendered as chip on every rate	Real feed replaces stub source
Quote ID + expiry countdown	Firm quotes only (bank RFQ response)	Rendered on booking flow	Real bank API replaces stub

The honest correction. There is no “Bloomberg deposit-rate feed for Meridian Bank plc.” Deposit rates are relationship-negotiated; every named bank rate API (JPM, BofA, HSBC, Citi, DB, DBS, StanChart, Barclays, Swedbank) is FX-only. The Register’s job is to remember the last-known rate and make override prominent.

Table C — Accounting Events

Field on screen	Real source in production	Current in demo	Step to make it real
Deal Executed event	Fusion Accounting Hub (AHCS) via OIC — XlaTrxH.csv/XlaTrxL.csv → Import Accounting Transactions ESS job	Stub event with source code, class, type, ledger id, GL date	Register as AHCS source system; wire OIC ERP Adapter
Deal Settled event	Same — AHCS via OIC	Same shape	Same
Daily accrual event	Same — AHCS via OIC (batch, end-of-day)	Nightly job writes stub events	Same
Deal Matured event	Same — AHCS via OIC	Same shape	Same
Internal transfer accounting	Fusion CM SLA — built-in “Bank Account Transfer Settled” event	Piggybacks on cashBank AccountTransfers stub	No AHCS needed — CM ships the events
Period status (open / closed)	Fusion GL business event — R13GLPeriodOpen / R13GLPeriodClose	Static “Sep-2026 open” in header	OIC subscribe trigger on GL business event
ESS request id	Returned by erpintegrations submitESSJobRequest immediately	Stub request id assigned on publish	Real erpintegrations invoke
GL journal batch id	Assigned asynchronously after Create Accounting + Import completes	Populates after 3s delay to show async	Poll getESSJobStatus ; GET journalBatches
Drill-back to deal	XLA source-transaction entities → distribution links → GL import references	Link in demo opens the deal in Register	Standard AHCS drill-back once wired

*The persistent myth we deliberately don't repeat. **journalBatches** REST is GET / PATCH / DELETE only — no POST exists per Oracle's 26C spec. Every write to GL or SLA funnels through the file-based **erpintegrations** endpoint. Any architect proposing “POST /journalBatches” is wrong; we don't design against it.*

Panel Q&A

The specific questions raised on the Sep 7 call, with the honest answers.

Goutham — Where does the cash pooling data come from?

Oracle Fusion Cash Management REST — **cashPools**, **cashPoolMembers**, **cashBankAccountTransfers**. Physical sweeps are fully executable via these resources. Notional pooling is a **cashPools** definition change — no money moves, it's a bank-side interest arrangement.

Anil — And bank balances? Statements?

No REST from Oracle. Only BICC extract, BI Publisher, or FBDI. That's a gap in Oracle's surface, not ours. We render the balance with a 15-minute as-of timestamp to be honest about staleness.

Goutham — Cash positions and forecasts?

Also no REST — Essbase cube only. Oracle's Release 26C added their own Cash Processing Agent AI for positioning, so we position the Register as **placement decisions**, not positioning. EPM plans. Register controls. Fusion moves.

Anil — Where do the deposit rates come from?

Deposit rates don't have a market-data feed. Bloomberg and LSEG publish indicative FX composites (BGN, BGNE) that are explicitly “not for trading.” Counterparty deposit rates come from the bank portal or dealer chat. Every named bank API is FX-only. So we pre-fill with the last-known negotiated rate for the counterparty and make override prominent — that is the real workflow.

Goutham — So for FX at least, is it Bloomberg live?

For pre-fill, yes — Bloomberg BGN via B-PIPE, or LSEG Real-Time via WebSocket. That's an indicative mid, not a firm quote. The firm rate on a ticket comes from the bank's RFQ/booking API (DBS FX Booking, BofA Guaranteed FX Rates) or a multi-bank platform (360T, FXall, FXGO) — carrying a quote id and short expiry.

Anil — How do the accounting events reach Oracle GL?

Oracle Fusion Accounting Hub via Oracle Integration Cloud. The Register registers as a source system with its own event classes (Deals, Settlements, Accruals). We hand Oracle raw transaction data as

XlaTrxH.csv/XlaTrxL.csv, SLA applies our rules, creates the subledger journal, posts to GL, and preserves drill-back from the GL balance to the deal.

Goutham — Why not just POST to journalBatches REST directly?

journalBatches is GET / PATCH / DELETE only. There is no POST in Oracle's 26C spec. Every write funnels through the file-based **erpintegrations** endpoint. It's a persistent architect myth — we deliberately don't use it.

Anil — At which lifecycle stages does the Register fire an accounting event?

Four — execution (trade date), settlement (value date), daily accrual (batch), and maturity. Proposed / approved / confirmed / closed are non-accountable events. This matches Oracle's own AHCS loan reference model.

Goutham — Does the Register listen back to Fusion for anything?

Only via OIC ERP Cloud Adapter subscribe trigger — Oracle states this verbatim, no third-party platform. Period events (**R13GLPeriodOpen** / **R13GLPeriodClose**) are reliable. Journal-posted and payment-settled events exist but are off by default and capped around 1000 events per hour.

Anil — Six checks — where do the reference data come from?

Counterparty master and limits from Fusion Cash Management (**cashCounterparties**, **cashLimits**) — stubbed today. Instrument permissions and tenor bands are policy data held in the Register itself. Concentration is computed on the Register's own book. Each check row in the UI is labelled with its source.

Goutham — How real is any of this — is anything actually integrated?

Nothing yet. It's a prototype demo. Every field on screen names the real endpoint it would come from, and every value is flagged as stubbed. The point is that the story is defensible and the build order from here to production is a straight line.

Anil — How is this different from Oracle's own Cash Processing Agent?

Oracle's agent does **positioning** — where is the cash right now, what does the forecast look like. The Register does **placement** — given a surplus, which permitted counterparty and instrument, subject to six checks. Different job. Register sits above Fusion, not against it.

Path to production

Rough order for a first working slice — 8 to 12 weeks.

1. Stand up OIC and the ERP Cloud Adapter.
2. Register the Register as an AHCS source system; define event classes (Deals, Settlements, Accruals) and accounting rules.
3. Wire the four lifecycle events (execute, settle, accrue, mature) to **erpintegrations** via OIC.
4. Wire **cashBankAccountTransfers** for internal transfers; poll status on 30s cadence.
5. BICC extract on 15-minute cadence for bank balances into a Register cache.
6. Bloomberg B-PIPE or LSEG Real-Time contract for FX indicative pre-fill.
7. Deposit rates: negotiated-rate memory in Register; treasurer overrides on ticket (this stays manual).

Not in scope for the first slice: bank statement reconciliation (BICC/FBDI), cash position/forecast (Essbase — EPM territory), firm-rate booking via a bank RFQ API (later phase, needs bank-side contract).

Three myths we deliberately don't repeat

Because these are the things a Fusion- or treasury-savvy reader will look for.

- “**POST to journalBatches REST to create a journal.**” No POST exists in Oracle 26C. Every write goes through **erpintegrations**.
- “**Bloomberg publishes counterparty deposit rates.**” Indicative FX composites only, explicitly “not for trading.” Deposit rates are portal or dealer.

- **“Fusion has a webhook for GL journal posted.”** Subscription is only via OIC ERP Adapter, capped ~1000 events/hour, most events off by default.

End of note. The prototype itself is the working demo of every row in the tables above; this document is the map from what's on screen to what would be real.